

Java Event Planner

Relatório do Projeto Final

SCC0204 — Programação Orientada a Objetos ICMC — USP

Integrante	Nº USP
Fabio Kauê Araujo da Silva	16311045
Kainã Alves Tureso	15466391
Luís Henrique de Queiroz Veras	14592414

Repositório: https://github.com/kainaas/Trabalho_P00

1 Requisitos

O **Java Event Planner** é um aplicativo *desktop* (Java + Swing) para organização de compromissos em um calendário. **Requisitos funcionais:**

- Cadastro, edição e exclusão de eventos com título, data, hora, duração, local, descrição e categoria (reunião, aniversário, compromisso ou outro).
- Eventos recorrentes (diário, semanal ou mensal), com a opção de editar/excluir *somente uma ocorrência* ou *a série inteira*.
- Participantes (nome e e-mail) e lembretes (10 min, 1 h ou 1 dia antes), exibidos em tempo de execução.
- Busca por palavra-chave (título, descrição, local ou categoria).
- Persistência dos eventos em arquivo, recarregados na abertura do programa.

Requisitos adicionais levantados pela nossa implementação:

- **Quatro modos de visualização** — Dia (agenda por hora), Semana (sete colunas), Mês (grade) e Ano (os doze meses) — trocados na barra superior.
- **Tema claro/escuro** aplicado a toda a interface, com ícones próprios para cada tema e botões totalmente repintados (sem “faixas brancas”).
- **Indicador da quantidade de eventos** de cada dia, na própria célula.
- **Seletor de data por calendário** no formulário de evento.
- **Robustez na leitura:** linhas inválidas do arquivo são ignoradas.

2 Descrição do Projeto

A aplicação segue o padrão **MVC**, refletido na divisão em pacotes: **Model** (dados e regras: `CalendarModel`, `Event`, `Attendee`, `Recurrence`, `EventStorage`, `AbstractModel` e as exceções), **View** (telas Swing: `CalendarView`, `TopPanel`, os quatro modos `DayViewPanel`/`WeekViewPanel`/`CalendarMonthPanel`/`YearViewPanel`, `DayEventsPanel`, `EventFormDialog`, `DatePickerDialog`, `ReminderService`, `Icons`, `ThemeColors`, `ViewMode`), **Controller** (`Controller`) e **Main**.

Observer. `CalendarModel` herda de `AbstractModel`, que encapsula um `PropertyChangeSupport`. Toda alteração relevante (novo evento, troca de data, de modo de visualização ou de tema) dispara uma notificação; a `CalendarView` implementa `PropertyChangeListener` e redesenha os painéis, sem que o modelo conheça a interface.

Visualizações. O centro da janela troca o painel conforme o `ViewMode` atual (interface `CalendarSubview`). **Recorrência:** guarda-se apenas o evento “mestre” e o método `occursOn(dia)` calcula sob demanda se ele cai em uma data, usando um conjunto de *datas de exceção* para remoções pontuais. O diagrama de classes está em `docs/uml.puml` e a documentação gerada (`javadoc`) em `docs/javadoc`.

3 Comentários sobre o Código

O código está documentado no padrão **Javadoc** (em inglês) nas classes e métodos não triviais. Conceitos de POO aplicados:

- **Encapsulamento:** campos `private` expostos por *getters/setters*; a validação de um `Event` é centralizada no `Controller`.
- **Herança:** `CalendarModel` de `AbstractModel`; as exceções de `Exception`; os painéis de `JPanel`, a janela de `JFrame` e os formulários de `JDialog`.

- **Polimorfismo:** `toString()` sobrescrito em `Recurrence`, `Attendee` e `ViewMode`; `CalendarView` implementa `PropertyChangeListener`; `ReminderService` implementa `Runnable`; os quatro modos implementam `CalendarSubview`.
- **Composição:** um `CalendarModel` é dono da lista de `Event`, e cada `Event` é dono de seus `Attendee`.

As cores de cada tema ficam em `ThemeColors`, cujo método `styleButton` repinta os botões de forma consistente mesmo no *Look and Feel* padrão (Metal), evitando as “faixas brancas” no modo escuro.

4 Plano de Testes

A lógica de negócio é coberta por uma suíte automatizada em **JUnit 5** (`test/`, 28 casos), executável com `test/run-tests.cmd`:

- `EventTest` — `occursOn` para os quatro tipos de recorrência, datas de exceção e cálculo do fim do evento.
- `CalendarModelTest` — título único, contagem por dia, ordenação cronológica, busca e remoção de ocorrência.
- `ControllerTest` — validações (título vazio, duração ≤ 0 , data inválida), bloqueio de duplicata e gravação em disco.
- `EventStorageTest` — ida-e-volta de gravação/leitura, tolerância a linhas corrompidas e arquivo inexistente.
- `ModelValueTest` — `toString` de `Attendee` e `Recurrence`.

Complementarmente, foram feitos testes manuais de interface: troca de tema, troca de modo de visualização, lembretes e o seletor de data por calendário.

5 Resultados dos Testes

Todos os 28 testes passam (Java 25, JUnit 5.8.2). Saída resumida do console:

```
[ 28 tests found ] [ 28 tests started ] [ 28 tests successful ] [ 0 tests failed ]
```

Os testes manuais de interface também passaram: o tema é aplicado a botões, ícones, células e campos (sem “faixas brancas”); os quatro modos de visualização funcionam; a persistência é estável e um arquivo corrompido não derruba o programa.

6 Procedimentos de Build

Não há bibliotecas externas — apenas o JDK (testado no Java 25; JDK 17+ deve funcionar).

1. **Instalar o JDK** (ex.: Eclipse Temurin, <https://adoptium.net>); verifique com `java -version` e `javac -version`.
2. **Obter o código:** `git clone https://github.com/kainaas/Trabalho_P00.git` e `cd Trabalho_P00/src`.
3. **Compilar** (Windows: `compile.cmd`): `javac -encoding UTF-8 -d bin ./Controller/*.java ./Model/*.java ./View/*.java ./Main/*.java`
4. **Executar** (Windows: `Run.cmd`): `cd bin && java Main.Main`. O diretório de trabalho deve ser `bin` para resolver os ícones em `bin/iconImages`; `events.txt` é criado ali no primeiro salvamento.
5. **Testes (opcional):** com `junit-platform-console-standalone-1.8.2.jar` na raiz, rode `test/run-tests.cmd`.

7 Problemas

- **Recorrência.** A primeira ideia (uma cópia por dia) gastava memória; a solução foi o evento “mestre” com `occursOn` sob demanda e um conjunto de datas de exceção.
- **Título único vs. editar uma ocorrência.** A regra de título único conflitava com a edição de “somente esta” ocorrência (que gera um avulso de mesmo título); resolvido com `addEventForced`, usado só nesse caso.
- **Formato do arquivo.** Persistir listas exigiu separadores em dois níveis (TAB e vírgula) e leitura tolerante a linhas inválidas.
- **Concorrência.** O `ReminderService` roda em uma *thread daemon* e os avisos usam `SwingUtilities.invokeLater`.

8 Comentários

O projeto foi desenvolvido de forma incremental e versionado no GitHub. A arquitetura MVC com Observer manteve a interface desacoplada das regras de negócio, o que facilitou adicionar recursos (tema escuro, indicador de eventos, modos de visualização e seletor de data) sem reescrever as telas. Conforme o *peer review* da disciplina, os apontamentos do grupo revisor serão documentados e respondidos nesta seção na versão final.